

s-t Path with Consecutive Red Vertices

Background knowledge and key points for the CS 6515 homework problem and Format Quiz

Part I — 1. Reading the problem

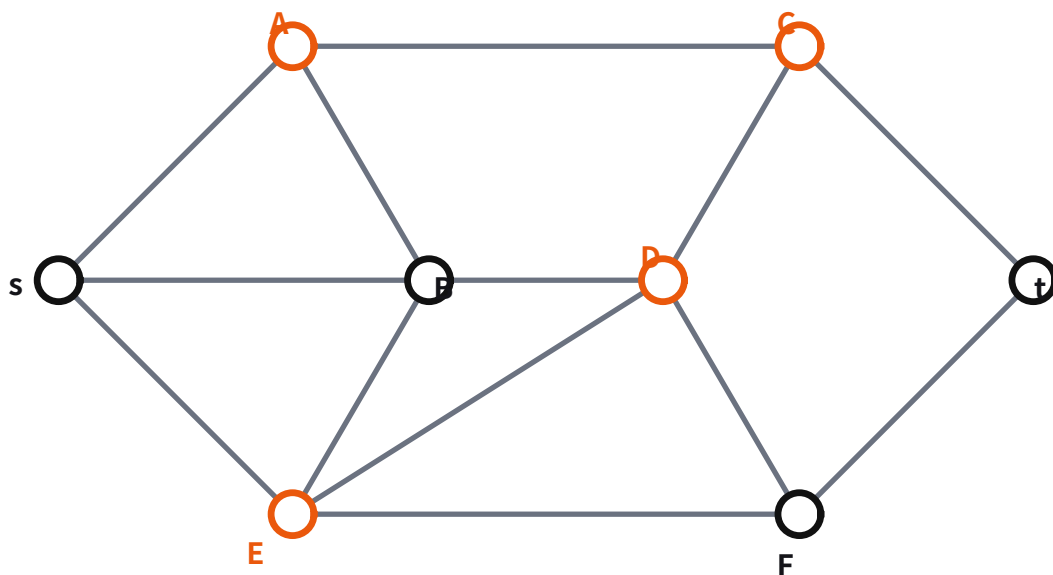
Input.

An undirected graph $G = (V, E)$. Every vertex is colored red or black, and its color can be accessed through $\text{col}(v)$ in $O(1)$ time. Two vertices s and t are given, and **both s and t are black**.

Decision.

Return TRUE if there exists a path from s to t such that all red vertices traversed by the path appear **consecutively** on it; FALSE otherwise.

A valid path must look like **black* · red+ · black*****, or **black***** (zero reds). Reds split into two blocks make it invalid.



The homework example graph. Orange circles = red vertices (A, C, D, E); black circles = black vertices (s, B, F, t).

Path examples.

Valid ✓ $s \rightarrow B \rightarrow D \rightarrow C \rightarrow t$ → reds (D, C) form one block.

Valid ✓ $s \rightarrow A \rightarrow C \rightarrow D \rightarrow F \rightarrow t$ → reds (A, C, D) form one block.

Invalid ✗ $s \rightarrow A \rightarrow B \rightarrow E \rightarrow D \rightarrow F \rightarrow t$ → reds split into A and (E, D) (the walk exists, but it violates the condition).

Part I — 2. Background A: graph, path, reachability

- **Undirected graph:** edges have no direction, so a $u \rightarrow v$ edge can be traversed both ways.
- **Walk:** a sequence of moves that may repeat vertices. **Path:** a walk with no repeated vertices. For reachability it suffices to think about walks — if a walk exists, a simple path exists too (just cut out cycles).

- **Reachability:** “can we get from s to t ?” — the final shape of this problem's question. If the condition (“reds consecutive”) can be **translated into graph structure**, what remains is a reachability problem.

Part I — 3. Background B: BFS/DFS is the basic tool for reachability

- Breadth-first search (BFS) / depth-first search (DFS) finds every vertex reachable from a start vertex in $O(V + E)$ time.
- On a **directed graph**, BFS/DFS simply follows edge directions, still in $O(V + E)$. The layered graph below is directed, so keep this in mind.
- Key insight: however complicated a condition looks, if we can build a **new graph whose edges are exactly the moves satisfying the condition**, the problem becomes “is t reachable from s in the new graph?”

Part I — 4. Background C: state augmentation

The condition “red vertices are consecutive” must be judged by looking at the **whole path**. BFS/DFS only sees the current vertex, so we need a **memory**: “am I inside the red block right now?”

State augmentation encodes this memory into the graph itself. Each vertex v is expanded into several copies (v , state), one per state, and only allowed moves become edges. Then “a path satisfying the condition” = “a path in the new graph (respecting directions)”.

The three states for this problem.

1. Not yet entered red — only black vertices so far
2. Inside the red block — currently traversing reds consecutively
3. Left the red block — the red block is over, only black from here

State transitions go **only $0 \rightarrow 1 \rightarrow 2$, one direction**. Once you leave the red block (2), you can never return to red (1), so the reds automatically form at most one consecutive block.

Part I — 5. Why naive approaches fail

Naive approach	Why it fails
Keep only black vertices, check s - t connectivity	In the example there is no s - t path using black vertices only (B's only black neighbor is s). This misses cases where reds are required.
Delete all red vertices, then check	Same reason — it discards valid paths that need a red block.
Enumerate all s - t paths and test each	The number of paths can be exponential, so this never finishes in polynomial time.
DFS to find one path, test it, backtrack on failure	Exponential in the worst case. “Reds consecutive” is a global condition: a greedily chosen path may get stuck while another path is valid.

So reds can neither be excluded entirely nor allowed without restriction. We need a way to test the global condition “exactly one consecutive block” in polynomial time → state augmentation is the right direction.

Part I — 6. Key idea: copy each vertex into 3 layers

Build a new graph H . Each vertex v gets one copy per state:

- **Layer 0** (v , 0): before entering red — **black vertices only**
- **Layer 1** (v , 1): inside the red block — **red vertices only**

- **Layer 2 (v, 2):** after leaving red — **black vertices only**

Edge rules (for each original edge $u-v$):

Original edge type	Edges added to H	Meaning
black-black	$(u,0)-(v,0)$ undirected, $(u,2)-(v,2)$ undirected	Black moves before/after the red block
black(u)-red(v)	$(u,0) \rightarrow (v,1)$ directed	Entering the red block (no way back)
red-red	$(u,1)-(v,1)$ undirected	Moves inside the red block
red(u)-black(v)	$(u,1) \rightarrow (v,2)$ directed	Leaving the red block (no way back)

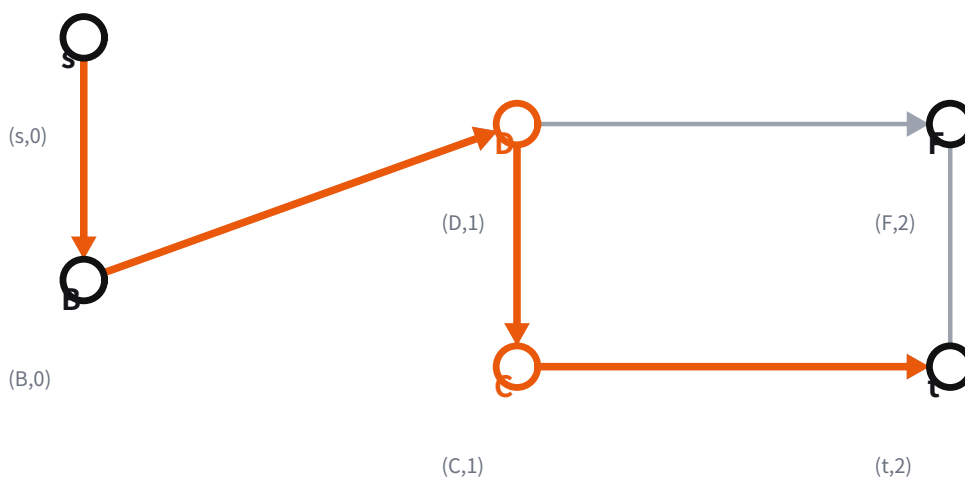
Direction is the key. Cross-layer edges must be directed so that the order “ $0 \rightarrow 1 \rightarrow 2$ ” is forced, which makes the reds (layer-1 vertices) one consecutive block on the path. Within a layer the graph is undirected, so moves are bidirectional.

Query: starting from $(s, 0)$, is $(t, 0)$ or $(t, 2)$ reachable by directed BFS/DFS? Reaching $(t, 0)$ = a valid path with no reds; reaching $(t, 2)$ = a valid path containing a red block.

Layer 0 · before red

Layer 1 · red block

Layer 2 · after red



Part of the example graph unfolded into layers. Orange arrows = the walk for the valid path $s-B-D-C-t$: $(s,0) \rightarrow (B,0) \rightarrow (D,1) \rightarrow (C,1) \rightarrow (t,2)$. Cross-layer moves are directed, so there is no way back into the red block.

Part I — 7. Example trace: $s-B-D-C-t$

1. Start at $(s, 0)$. s is black $\rightarrow$ layer 0.
2. $s-B$ is a black-black edge $\rightarrow$ move within layer 0: $(B, 0)$.
3. $B-D$ is a black-red edge $\rightarrow$ directed entry: $(D, 1)$. Now inside the red block.
4. $D-C$ is a red-red edge $\rightarrow$ move within layer 1: $(C, 1)$.
5. $C-t$ is a red-black edge $\rightarrow$ directed exit: $(t, 2)$. Reached!

The red vertices D, C on the path correspond exactly to the consecutive segment spent in layer 1. In contrast, the invalid path $s-A-B-E-D-F-t$ cannot be replayed in the layered graph: after entering $(A,1)$, there is **no edge back** to $(B,0)$.

Check by hand.

Tracing $s-A-C-D-F-t$ in the layered graph gives $(s,0) \rightarrow (A,1) \rightarrow (C,1) \rightarrow (D,1) \rightarrow (F,2) \rightarrow (t,2)$, reaching $(t,2)$. The

reds (A, C, D) sit together in layer 1.

Part I — 8. Correctness & runtime checkpoints

Per Ed #385, the submitted answer must be narrative **(a)/(b)/(c)** prose (no pseudocode).

(a) Algorithm — what to build

- Input G , color function col , s , t (black). Build the layered graph H : at most $3V$ vertices, and each original edge contributes at most 3 edges plus cross-layer directed edges, so H has $O(V + E)$ size.
- Run directed BFS/DFS from $(s, 0)$; return whether $(t, 0)$ or $(t, 2)$ is reachable.

(b) Correctness — why it works (both directions)

- ($\Rightarrow$) If a valid path $P = \text{black}^* \cdot \text{red}^* \cdot \text{black}^*$ exists, lifting each vertex to its block's layer yields a walk in H .
- ($\Leftarrow$) If H has a walk $(s, 0) \rightarrow (t, 0)$ or $(t, 2)$, projecting it to G gives a walk whose red vertices come only from the consecutive layer-1 segment — hence consecutive. A walk can be shortcut into a simple path by cutting cycles.

(c) Runtime analysis

- H has $\leq 3n$ vertices and $O(m)$ edges $\rightarrow$ construction $O(n + m)$.
- Directed BFS/DFS costs $O(V' + E') = O(n + m)$.
- $\text{col}(v)$ is $O(1)$. Overall $O(n + m)$.

Part I — 9. Pitfall checklist

- Did you use the fact that **s and t are black**? It pins the start to $(s, 0)$ and the targets to $(t, 0)/(t, 2)$.
- Did you handle the **zero-red** case? Reachability of $(t, 0)$ must also be checked.
- When copying undirected edges into layers: **undirected within a layer, directed between layers**?
- Layer 1 holds red vertices only; layers 0/2 hold black vertices only?
- In correctness, did you include the “walk $\rightarrow$ path” step? (One line on cycle-cutting.)
- In the runtime, did you state that H is a constant factor larger than the input?

Part I — 10. Extension (UNGRADED) hints

Even when s and t can be **any color**, the same layered graph works. Only the start and target layers change.

- **Start:** if s is black start at $(s, 0)$; if red start at **$(s, 1)$** .
- **Targets:** if t is black check $(t, 0)$ or $(t, 2)$. If t is red also accept **$(t, 1)$** .
- If s and t are both red and the whole path is red, the walk stays in layer 1 and reaches $(t, 1)$.
- The crux is unchanged: **the layer structure enforces the condition (reds consecutive); only the start/target layers adapt.**

Part I — 11. Try by hand

- Draw the example graph on paper and write the layer number (0/1/2) next to each vertex of $s\text{--}B\text{--}D\text{--}C\text{--}t$.
- Pinpoint where the invalid path $s\text{--}A\text{--}B\text{--}E\text{--}D\text{--}F\text{--}t$ stops being traceable in the layered graph.
- Invent a small 4-vertex graph, draw its layered graph, and run BFS by hand.
- Extension: make a tiny example with red s and repeat the process starting from layer $(s, 1)$.

Part II — Format Quiz: "Avoiding x"

This part covers the class Format Quiz on Graph Algorithms together with the homework. The quiz's real goal is not picking answers but training the **answer format: (a) algorithm / (b) correctness justification / (c) runtime analysis**.

Three rules the quiz enforces.

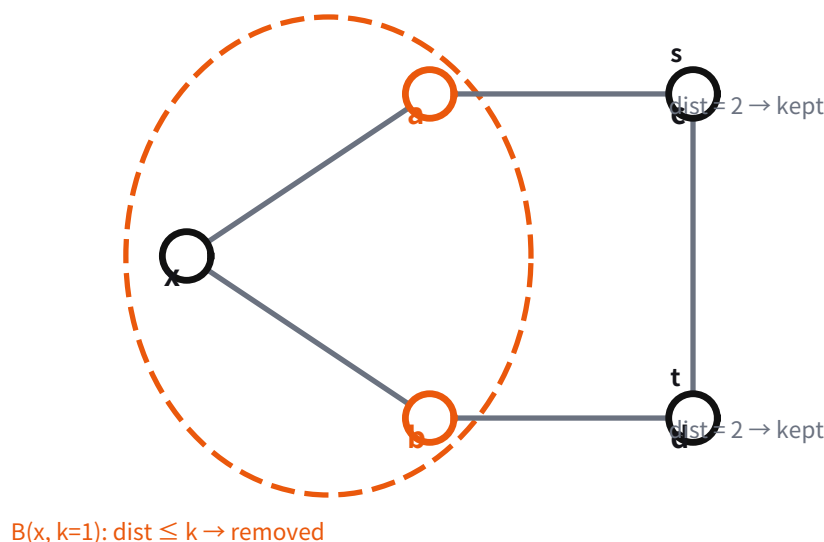
1. **Do not modify black boxes.** Use DFS, BFS, Dijkstra's as-is, and **modify the input (graph)** so the black box finds the answer. Modifying a black box is penalized even if the solution works.
2. **(b) justification** must explain why the algorithm works — in particular **why the input was modified that way**.
3. **(c) runtime** must **show the calculation**, not just the final value. A known black box's runtime may be stated, but the input-modification cost plus each black box's runtime must be summed term by term.

Q1. The problem "Avoiding x"

Input.

An unweighted, undirected graph $G = (V, E)$ (adjacency list). A vertex x and a value k are given. Find a **shortest path** from s to t that **avoids x and every vertex within distance $\leq k$ of x** . Assume such a path exists.

Background. Running BFS from x gives $\text{dist}[v]$ = the shortest-path distance from x for every vertex v . The vertices with $\text{dist}[v] \leq k$ are exactly the **ball** $B(x, k)$ of radius k around x . Deleting them from the graph means every s - t path in the remaining graph G' automatically avoids the forbidden vertices.



BFS from x computes dist . Deleting vertices with $\text{dist} \leq k$ (a, b) makes the s - t shortest path (c - d) in the remaining graph automatically valid.

Q2. The four candidate solutions a-d

(a)

BFS from x to get $\text{dist} \rightarrow$ *modify BFS* to skip any vertex with $\text{dist} \leq k$ when it would be visited $\rightarrow$ run the

modified BFS from s , return the shortest path.

→ Works, but **modifies the BFS black box** — violates course discipline. Penalized, no full credit.

(b)

BFS from x to get dist → *delete* all vertices with $\text{dist} \leq k$ from the graph → BFS from s on G' , return the shortest path.

→ Uses black boxes as-is. Correct + $O(n+m)$. Optimal.

(c) — **the pseudocode version of (b)**

$\text{dist} = \text{BFS}(G, x).\text{dist}$ → delete vertices with $\text{dist} \leq k$ → $\text{BFS}(G', s)$ → rebuild the path by walking the prev array back from t .

→ Correct + optimal. The most complete form, explicitly rebuilding the path.

```
dist = BFS(G, x).dist
G' = G
for v in V:
    if dist[v] <= k: G'.remove(v)
prev = BFS(G', s).prev
path_list = t; v = t
while prev[v] not null:
    v = prev[v]; path_list = v + path_list
return path_list
```

(d)

Assume all edge costs are 1 → Dijkstra's from x to get dist → set the cost of edges touching vertices with $\text{dist} \leq k$ to infinite → Dijkstra's from s on G' .

→ Correct but **$O((n+m) \log n)$** . Dijkstra's is overkill on an unweighted graph — BFS suffices. Not optimal.

Q3. Q1 analysis — picking "correct and optimal"

- **Correctness:** all four solutions work ("the above solutions are all correct in some way").
- **Optimality:** reading the input already costs $\Omega(n+m)$, so **$O(n+m)$ is optimal**. (d) costs $O((n+m) \log n)$ → out.
- **Black-box discipline:** (a) modifies BFS → penalized → no full credit.
- **Conclusion: (b) and (c) are correct and optimal.** The stimulus itself says "pick any that would receive full credit". (c) is the most complete statement, including path reconstruction.

Exam point. "It works" and "it receives full credit" are different. Modifying a black box and using an unnecessarily heavy Dijkstra's are classic point-losers.

Q4. Q2 matching — correctness justifications

Solution	Matching justification	Why
(a)	**6.** "By *modifying BFS* to avoid unwanted vertices, ..."	Matches (a), which modifies BFS itself.
(b)	**3.** "By *modifying the graph* to remove any unwanted vertices, ..."	Matches (b), which modifies the graph. Core logic: since the vertices were deleted, every path BFS finds automatically avoids the forbidden vertices.
(c)	**1.** "Our *code* removes any vertices ... follow the *prev array* from t to s ..."	Matches (c) as written in code ("Our code", "prev array").

Solution	Matching justification	Why
(d)	**4.** “... make edges ... *cost infinite*, *Dijkstra's* will avoid them ...”	Matches (d) with infinite costs + Dijkstra's.

Distractors: 2. “No justification needed since it's already explained” — directly violates Ed #385 (justification is mandatory). 5. “The TA said it would work” — not a justification.

Point: a good justification explains “**why the input was modified**”. The key sentence connects the modification to the black box's output: because of the modification, whatever the black box returns is exactly what we want.

Q5. Q3 matching — runtime analyses

Solution	Matching runtime	Why
(a)	**R1.** First BFS $O(n+m)$ + modified BFS still $O(n+m) \rightarrow O(n+m)$	Mentions both BFS runs.
(b)	**R2.** First BFS $O(n+m)$ + deleting vertices/edges $O(n+m)$ + second BFS $O(n+m) \rightarrow O(n+m)$	Accounts for the deletion step.
(c)	**R3.** BFS(G, x) $O(n+m)$ + for v in V $O(n)$ + BFS(G', s) $O(n+m)$ + prev walk $O(n) = O(n+m)$	Term-by-term accounting of the code.
(d)	**R4.** Two rounds of Dijkstra's $O((n+m) \log n)$ + edge updates $O(n+m) \rightarrow O((n+m) \log n)$	Dijkstra's dominates.

Distractors: R5. “ $O(nm)$ ” — wrong value. R6. Bare “ $O(n+m)$ ” — no calculation shown (violates the (c) rule). R7. “BFS is $O(n+m)$ so the final runtime is $O(n+m)$ ” — ignores the other steps, incomplete.

Point: write runtimes as a **term-by-term sum**. A black box's known runtime may be stated; the input-modification cost (deletion loop, edge updates) must be computed explicitly.

Q6. Key points + connection to Part I (homework)

- **The “modify the graph + black box” pattern.** The Part I homework (build a layered graph, then BFS/DFS) and this quiz (delete forbidden vertices, then BFS) are the **same pattern**. It is the core GR-unit technique: translate the condition into graph structure, then run an unmodified black box.
- **Don't fix the black box.** Instead of opening up BFS as in (a), change the input graph. Layered graphs, vertex deletion, and edge-cost changes are all “input modification”.
- **Pick the right black box.** Unweighted $\rightarrow$ BFS, $O(n+m)$. Weighted $\rightarrow$ Dijkstra's, $O((n+m) \log n)$. Using Dijkstra's on an unweighted graph forfeits optimality.
- **Justification formula:** “Because the input was modified this way, what the black box returns is exactly what we want.”
- **Runtime formula:** $\Sigma(\text{modification costs}) + \Sigma(\text{black-box costs})$, stated term by term.

These are study notes. Per Ed #385, the submitted answer must be narrative (a)/(b)/(c) prose written by you. The quiz options and matchings here are for understanding, not a format to copy into an answer.